

Project Ascent: Building a Small Language Model from Scratch

Tokenizers, Adaptation, Retrieval, Prompting and a Tool-Using Agent

Project Ascent

October 2026

Abstract

Project Ascent builds a decoder-only Transformer language model end to end on a CPU-sized budget and extends it in four phases: a character-level baseline, a trained BPE tokenizer with architectural variants, LoRA fine-tuning plus lexical retrieval, and a prompt-chain agent with memory and a safe calculator. This report records what was built and what was measured. All trained models are tiny (about 1.1×10^5 parameters, 500 optimisation steps, TinyShakespeare, one seed). The only head-to-head comparison with real numbers is character versus BPE tokenization, where BPE reaches 3.09 versus 3.45 validation bits per character. Prompting, instruction-following and agent experiments verify the *pipelines*; they do not show useful model capability, and no such claim is made.

1 System overview

Component	Implementation
Model	Decoder-only MiniGPT : learned positions, tied embeddings, causal self-attention, LayerNorm or RMSNorm, PyTorch
Tokenizers	scaled-dot-product attention with a manual fallback Deterministic character tokenizer (<code><unk></code> = ID 0); deterministic character-initialised BPE with JSON-serialised merges
Decoding	Greedy, beam, <i>top-k</i> and <i>top-p</i>
Training	Argparse plus optional YAML, CPU/CUDA selection, checkpoints with optimizer state and resume compatibility checks
Adaptation	Alpaca-style formatting with prompt-loss masking; frozen-base LoRA on the attention <code>qkv</code> and output projections
Retrieval	Dependency-free lexical chunk retrieval with source provenance and bounded prompts
Agent	Memory window, <code>/calc</code> command, model-requested <code>CALC(...)</code> calls, bounded tool rounds, AST-based calculator (no <code>eval</code>)

Table 1: Implemented architecture. Every ticket has automated tests; the suite is run with `python -m unittest discover -s tests`.

2 Phase 1 and 3 smoke runs

These runs show that the plumbing works. They are not quality evidence.

Run	Configuration	Result
Baseline smoke	1 layer, 32 dim, 4 heads, 32 context, 8 steps	Validation loss 4.1424 $\rightarrow$ 3.9212; perplexity 62.96 $\rightarrow$ 50.46
LoRA smoke	3 instructions, rank 2, 6 steps	384 trainable parameters; final supervised loss 4.0325
RAG smoke	Query “Which planet is red?”	Retrieved <code>knowledge.txt</code> , lexical score 0.75; generated text was not an answer

Table 2: Smoke-test results. The undertrained baseline produced degenerate text; retrieval and provenance work, while answer quality would need a substantive training run.

3 Character versus BPE tokenization

Setup. Both models use TinyShakespeare, seed 42, 2 layers, 64 dimensions, 4 heads, context 64, batch size 16, learning rate 10^{-3} , 500 steps, CPU. BPE uses 100 merges.

Metric	Character	BPE
Vocabulary size	66	166
Parameters	108,416	114,816
Characters per token (validation)	1.000	1.620
Validation loss (per token)	2.3931	3.4757
Perplexity per token (not comparable)	10.95	32.32
Validation bits per character	3.4525	3.0946
Perplexity per character	10.95	8.54
Greedy repetition (char 3-gram)	0.970	0.742
Sampled repetition (char 3-gram)	0.126	0.091
Greedy / sampled characters per second	3429 / 3144	4148 / 4277

Table 3: Character versus BPE (single seed). Per-token perplexity is shown only to explain why it must not be compared: a BPE token carries 1.62 characters on average.

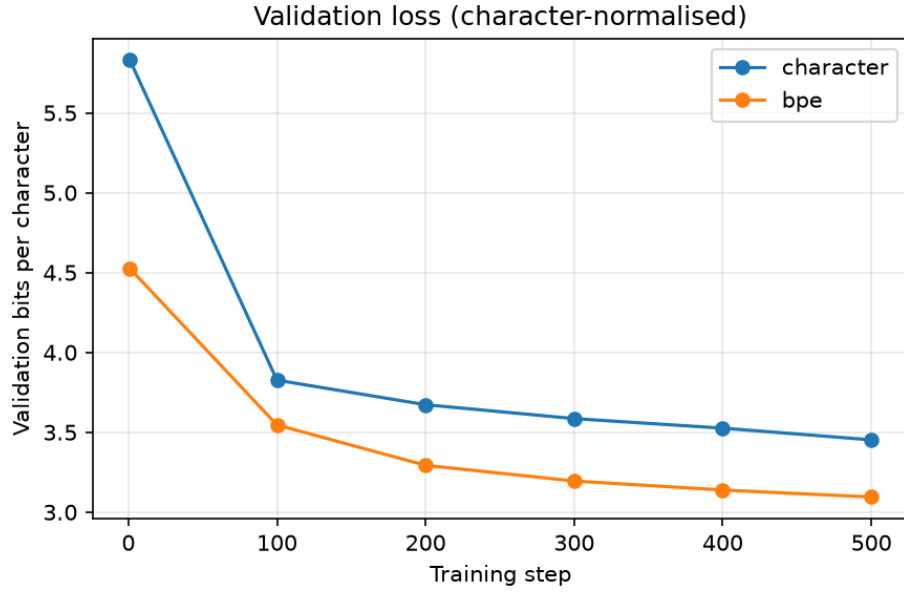


Figure 1: Validation loss converted to bits per character using each run’s measured characters-per-token ratio. BPE is lower at every evaluated step.

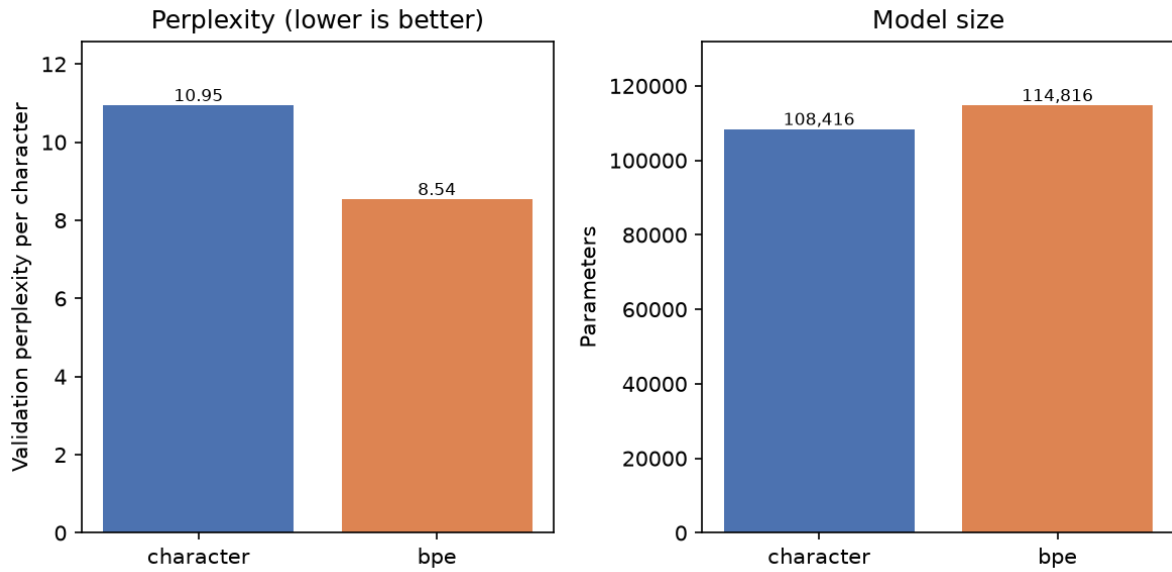


Figure 2: Per-character perplexity and parameter counts. BPE costs 6,400 extra parameters (5.9%), all from the 100 additional embedding rows.

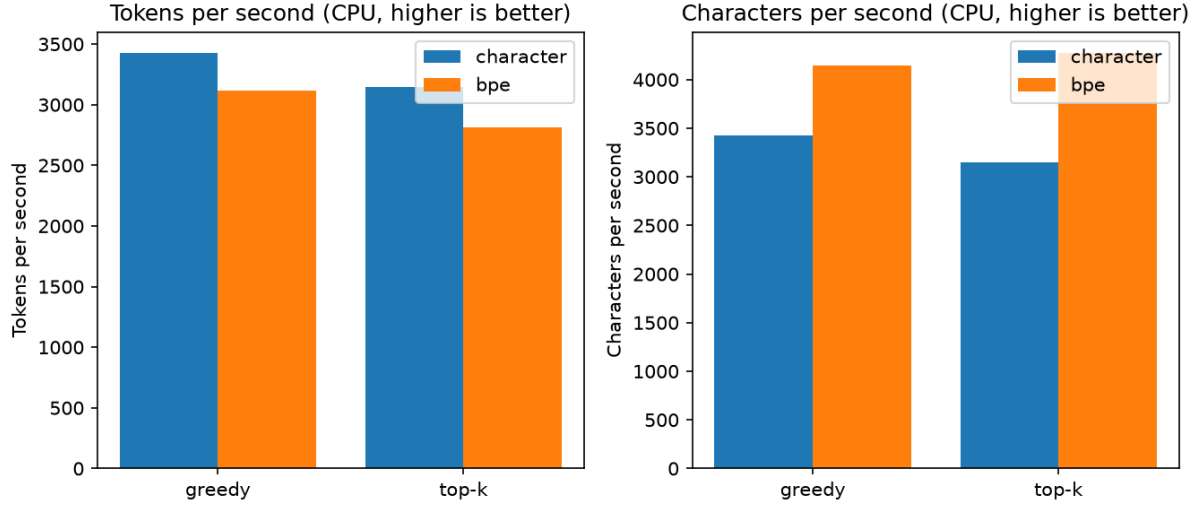


Figure 3: Generation speed on CPU from one 200-token sample per mode. BPE is slower per token but produces more characters per second because each step emits more than one character.

Interpretation. On the character-normalised scale BPE is about 10% better in bits per character and repeats less. Greedy output loops in both models. BPE samples contain more real words, but both models are heavily undertrained, so this is a relative signal only.

Confounds. (i) With a fixed 64-token context, BPE sees roughly 104 characters per window and about $1.6\times$ more text per step, so part of its advantage may come from data exposure rather than tokenization quality. (ii) BPE merges were learned on the full corpus, including validation text. (iii) Validation loss uses 20 random batches, not a full sweep. (iv) One seed and one merge count give no variance estimate. (v) The 28.2 s versus 5.0 s wall-clock figure includes pure-Python BPE training and encoding and is not a training-throughput comparison.

4 Attention inspection

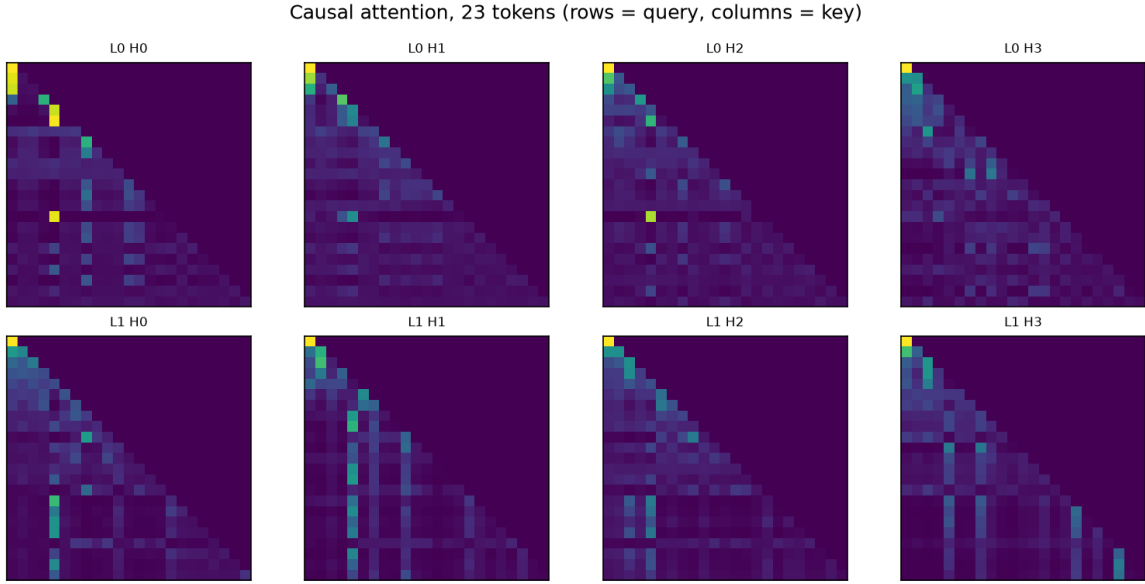


Figure 4: Per-head causal attention maps for the BPE model on a 23-token prompt (rows are queries, columns are keys). Every map is lower-triangular, as required by causal masking. Some heads show strong attention to the first token and a few vertical stripes. With 500 training steps this illustrates the extraction tooling; it is not evidence of learned linguistic structure.

5 Prompting

Zero-shot, few-shot (2 shots) and chain-of-thought prompts were run greedily on six arithmetic and factual tasks, with numbers spelled out because the corpus has almost no digits.

Checkpoint	Style	Exact / contains	Prompt truncated	Mean prompt tokens
BPE	zero-shot	0/6	0%	25.3
BPE	few-shot	0/6	100%	72.3
BPE	CoT	0/6	100%	174.3
Character	zero-shot	0/6	0%	37.5
Character	few-shot	0/6	100%	104.5
Character	CoT	0/6	100%	263.5

Table 4: Prompting results. All styles score zero, so no ranking of prompt styles is claimed.

The measurable effect is context budget: with a 64-token window every few-shot and CoT prompt is cropped, which removes the exemplars. BPE needs about $0.67\times$ the prompt tokens of the character model for the same text, a practical benefit for in-context prompting.

6 Agent: memory and calculator

The agent builds each prompt from a preamble and the newest conversation turns that fit a character budget, then optionally executes a calculator call and feeds the result back, for at most two rounds. The calculator parses with `ast` and whitelists numeric literals, `+` `-` `*` `/` `//`

% ** and unary signs. It limits expressions to 200 characters, nesting to depth 50, exponents to magnitude 100 and results to 10^{100} .

Input	Observed result
/calc (2+3)*4	20 (no model call)
/calc 2**1000	Rejected: exponent magnitude is limited to 100
/calc __import__('os')	Rejected: unsupported syntax (call)
Second process, same memory file	10 turns loaded (8 saved plus 2 new)

Table 5: Owner-run demo on the BPE checkpoint; 42 tests passed at that point.

Limitations. The 500-step models cannot emit `CALC(...)` themselves, so model-initiated tool use is verified only with scripted stubs; the working tool path is `/calc`. Digits are `<unk>` for the character model. Memory is a recency window, not summarisation or retrieval.

7 Limitations and conclusion

- All trained results are single-seed, 500-step runs of roughly 110k-parameter models on one small corpus.
- Instruction following and RAG answer quality were not evaluated beyond smoke runs; the retrieval and adaptation code paths are tested but their quality is unmeasured.
- Latency comes from single CPU samples and is indicative only.
- BPE is course-scale, not production byte-level BPE, and retrieval is lexical only.

The supported conclusions are narrow. The full pipeline works and is tested. BPE gives a roughly 10% lower bits-per-character than the character baseline in this setup, with the confounds listed above. The tool layer rejects unsafe expressions and persists memory. Stronger claims would need longer training, a larger context, multiple seeds and held-out instruction and RAG evaluations.